



DESAFÍO DE TRIPULACIONES 2026 :: EQUIPO PLANGUNE

Índice

1. Introducción	4
Estructura del repositorio	4
2. Resumen Ejecutivo	5
Hallazgos principales	5
3. Contexto y Usuarios	5
Público objetivo primario	5
Usuarios secundarios	5
Necesidades clave	6
4. Alcance Funcional y Requisitos	7
MVP funcional	6
Requisitos no funcionales	6
Requisitos de producto	6
5. Arquitectura y Diseño Técnico	7
Visión general	7
Componentes principales	7
Flujo de datos	8
Integración de servicios externos	8
Entorno de desarrollo	8
6. Modelo de Datos y Persistencia	8
Esquema Prisma	8
Entidad central: Event	9
Privacidad y datos sensibles	9
Operaciones de datos	9
7. Backend y API REST	9
Contrato público	9
Endpoints clave	10
Autenticación y autorización	10
Validación y errores	10
Recomendaciones híbridas	11
Asistente familiar	11
8. Frontend y Experiencia de Usuario	11
Estado actual	11
Estrategia UX	11
Integración técnica	11
Puntos pendientes	12
9. IA, Asistente Conversacional e Integraciones Externas	12
Arquitectura de IA	12
Contrato del asistente	12
Integración de Open-Meteo y Euskadi	12
Fallback y resiliencia	12
10. Seguridad, Privacidad y Compliance	13

Medidas aplicadas..... 13

Privacidad de menores..... 13

Riesgos de IA 14

Deuda de compliance..... 14

11. Calidad, Pruebas y Metodología 14

 Pruebas existentes 14

 Metodología..... 14

 Documentación relevante..... 14

12. Riesgos y Deuda Técnica 15

 Riesgos técnicos 15

 Deuda tecnológica..... 15

13. Roadmap y Próximos Pasos 15

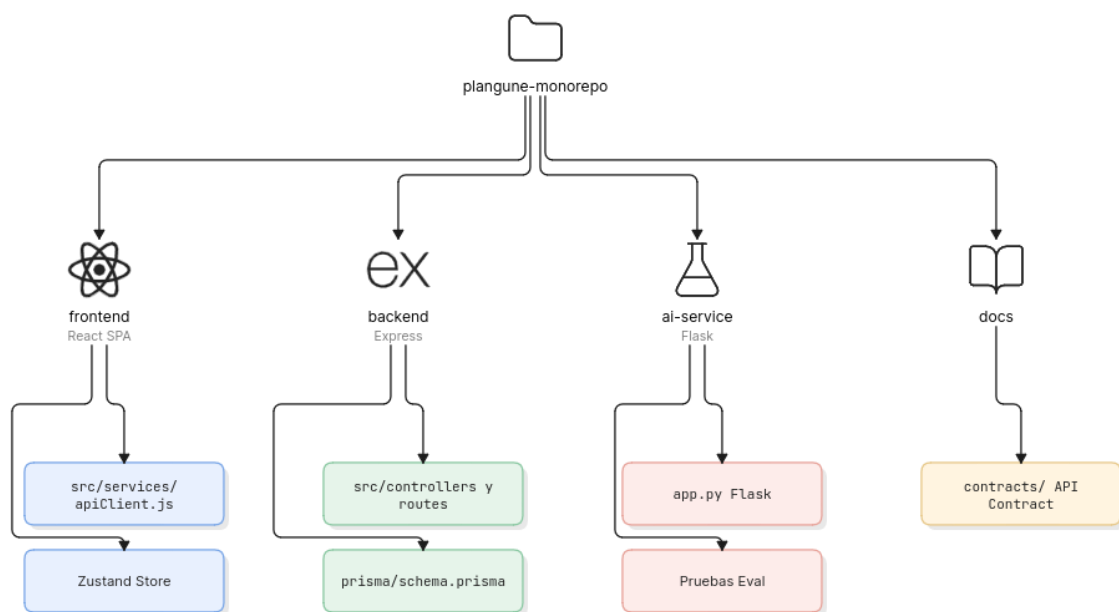
14. Referencias Internas 15

1. Introducción

Esta memoria sintetiza el estado actual del proyecto DESAFIO-26: el MVP fullstack preparado para el desafío de tripulaciones 2026. El producto provisional se conoce como Plangune, una plataforma responsive para familias con bebés y niños pequeños que desean encontrar planes, actividades y negocios familiares en Euskadi.

Estructura del repositorio

El repositorio es un monorepo npm con los siguientes dominios principales:



- ▶ frontend/: React + Vite SPA mobile-first.
- ▶ backend/: Node.js + Express API REST con arquitectura MVC.
- ▶ ai-service/: prototipo Python/Flask para asistente LLM local.
- ▶ docs/: documentación técnica, contratos API, auditorías y ADRs.

El objetivo de este documento es describir el producto, el estado técnico actual, las decisiones de diseño, los riesgos y la hoja de ruta inmediata.

2. Resumen Ejecutivo

DESAFIO-26 aborda una necesidad concreta: las familias con bebés y niños pequeños necesitan planes familiares en Euskadi que sean fáciles de filtrar, accesibles y confiables. El MVP actual ofrece:

- Catálogo de eventos con atributos prácticos para bebés y niños.
- Recomendador híbrido con Data API y fallback local explicable.
- Asistente conversacional demo con IA y degradación amable.
- Gestión de favoritos persistida en PostgreSQL.
- Una API pública consistente bajo /api.

Hallazgos principales

- La arquitectura está bien delimitada: el frontend consume solo el backend Express.
- El modelo de datos Prisma se alinea con el esquema real de docs/data/schema-real/init.sql.
- El asistente IA se habilita como opción y mantiene un fallback local seguro.
- La implementación contiene riesgos técnicos puntuales: contrato legacy activity/event, dependencia de servicios externos y el prototipo ai-service.
- El proyecto cumple un MVP funcional, pero requiere consolidación antes de producción.

3. Contexto y Usuarios

Público objetivo primario

- Familias jóvenes en Euskadi con bebés y niños pequeños.
- Visitantes que buscan planes familiares rápidos y bien filtrados.
- Padres que necesitan opciones con cambiador, carrito, interior o accesibilidad.

Usuarios secundarios

- Negocios familiares y actividades que desean publicar eventos u ofertas.
- Equipos internos de moderación, producto y datos.
- Desarrolladores responsables de la integración de IA y la contratación externa.

Necesidades clave

- Descubrir planes de ocio con filtros reales de familia.
- Recuperar experiencias indoor/covered cuando hace mal tiempo.
- Tener recomendaciones explicables y una UI que no falle con servicios externos.
- Garantizar confidencialidad y minimización de datos de menores.

4. Alcance Funcional y Requisitos

MVP funcional

- Descubrimiento de eventos con filtros avanzados (GET /api/events).
- Detalle de evento y atributos de accesibilidad.
- Favoritos persistidos por usuario.
- Recomendaciones híbridas con Data API y fallback local.
- Asistente familiar conversacional.
- Autenticación básica con roles family, business y admin.

Requisitos no funcionales

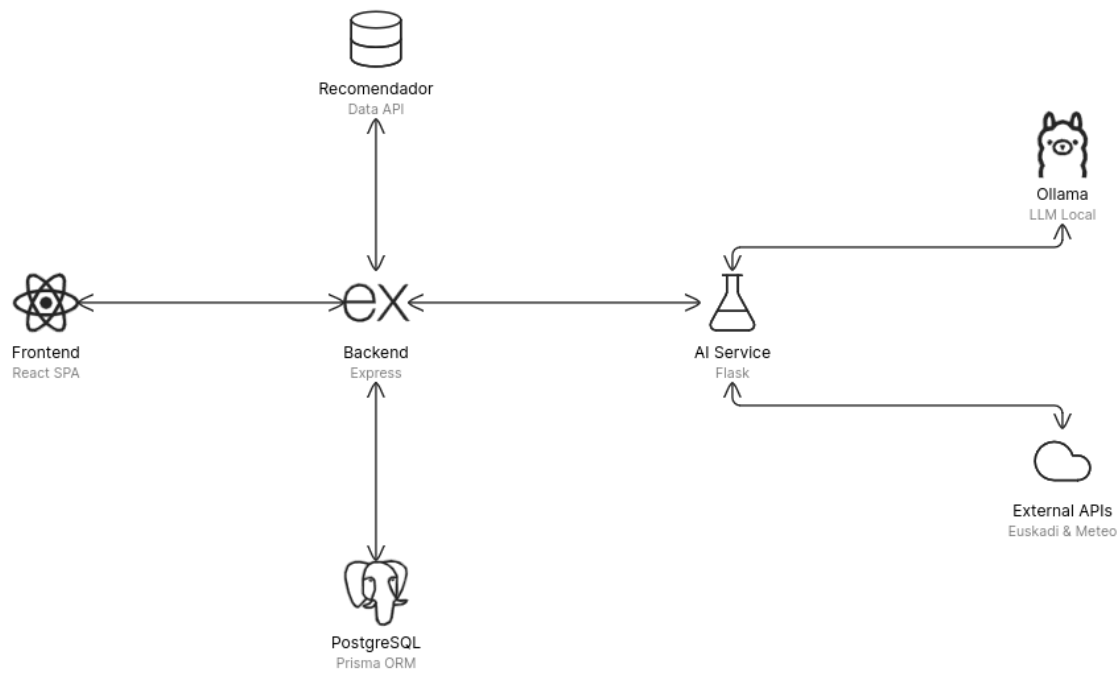
- Mobile-first y responsive.
- Facade única: el frontend solo consume la API Express.
- Fallback tolerante frente a fallos externos.
- Minimización de datos personales de menores.
- Desarrollo local reproducible con Docker y .env.example.
- Contrato API documentado en docs/contracts/frontend-backend-api-contract.md.

Requisitos de producto

- Filtrado por municipio, territorio, categoria, tipo_evento.
- Condiciones familiares: es_interior, es_carrito, es_cambiador, es_silla_ruedas, es_mascotas.
- Criterios de edad mínima y rango temporal.
- Recomendaciones con razones de score legibles.
- Asistente de familia que ofrezca respuesta en lenguaje natural y recomendaciones cuando sea posible.

5. Arquitectura y Diseño Técnico

Visión general



La solución se estructura como una aplicación cliente/servidor con dependencias externas opcionales. El frontend React se despliega como SPA y el backend Express actúa como fachada única frente a la base de datos y los servicios externos.

Componentes principales

Componente	Descripción
frontend/src/services/apiClient.js	Cliente Axios con VITE_API_URL.
recommendationsApi.js	Consumo de GET /api/recommendations.
familyChatApi.js	Consumo de POST /api/assistant/family-plan.
backend/src/routes/, controllers/, services/, repositories/	Capas MVC del backend.
backend/prisma/	Esquema Prisma, migraciones y seed.
ai-service/	Prototipo Flask e integración con Ollama/Open-Meteo.

Flujo de datos

1. El usuario interactúa con el frontend.
2. El frontend solicita datos al backend en /api.
3. El backend consulta PostgreSQL/Prisma o servicios externos.
4. El backend normaliza la respuesta y la devuelve al cliente.
5. El frontend muestra eventos, recomendaciones o respuestas del asistente.

Integración de servicios externos

Variable de entorno	Función
DATA_API_URL	Recomendador Data API (habilitable).
LLM_ASSISTANT_API_URL	Asistente LLM local vía ai-service.
CLOUD_ASSISTANT_*	Preparado como opción futura, aislado del frontend.
CLIENT_URL	Origen permitido en CORS.

Entorno de desarrollo

- compose.yaml orquesta backend, frontend y PostgreSQL.
- PostgreSQL local expuesto en localhost:5434.
- backend/.env.example documenta variables sensibles y habilitaciones.

6. Modelo de Datos y Persistencia

Esquema Prisma

El dominio se define en backend/prisma/schema.prisma con los siguientes modelos:

- ▶ User
- ▶ Family y FamilyMember
- ▶ Business
- ▶ Event
- ▶ Offer
- ▶ Plan y PlanEvent
- ▶ UserFavoriteEvent, UserFavoritePlan, UserSelectedRecommendation

El esquema Prisma mapea a las tablas reales con @@map y respeta la estructura de docs/data/schema-real/init.sql.

Entidad central: Event

Event es la entidad del catálogo de actividades. Sus atributos capturan información de ubicación y contacto, condiciones de accesibilidad, edad mínima, temporalidad, precio y presentación.

Los campos de accesibilidad son clave para el producto:

```
es_interior · es_carrito · es_cambiador · es_silla_ruedas ·  
es_mascotas
```

Privacidad y datos sensibles

- FamilyMember almacena solo age y name opcional.
- Business.nif es opcional y solo se registra si el negocio lo proporciona.
- El proyecto aplica minimización de datos, especialmente para menores.

Operaciones de datos

- Seed local con datos demo para usuarios, negocios y eventos.
- backend/prisma/import-events-from-csv.js para cargar eventos adicionales de forma controlada.
- Migraciones iniciales presentes en backend/prisma/migrations/.

7. Backend y API REST

Contrato público

El backend expone /api como única fachada. El contrato se documenta en docs/contracts/frontend-backend-api-contract.md y docs/api.md.

Endpoints clave

Endpoint	Descripción
GET /api/health	Estado del servicio.
GET /api/events	Listado de eventos con filtros.
GET /api/events/:id	Detalle de un evento.
GET /api/recommendations	Recomendaciones híbridas.
GET /api/favorites	Favoritos del usuario autenticado.
POST /api/favorites/:activityId	Añadir a favoritos.
DELETE /api/favorites/:activityId	Eliminar de favoritos.

Endpoint	Descripción
POST /api/assistant/family-plan	Asistente familiar conversacional.
POST /api/auth/register	Registro de usuario.
POST /api/auth/login	Login.
GET /api/auth/me	Perfil del usuario autenticado.
POST /api/auth/logout	Cierre de sesión.

Autenticación y autorización

- JWT firmados con JWT_SECRET.
- Cookies httpOnly para mayor seguridad.
- Roles: family, business y admin.
- Middlewares de autorización en favoritos y rutas protegidas.
- CORS restringido a CLIENT_URL.

Validación y errores

- express-validator para request validation.
- Código 422 en validación y 404 en recurso inexistente.
- Manejo centralizado de errores.

Recomendaciones híbridas

GET /api/recommendations retorna planes con campo source:

```
data-api      → cuando la Data API responde correctamente
local-fallback → cuando la capa externa no está disponible
```

La respuesta incluye: event, activity, score, reasons y source.

Asistente familiar

POST /api/assistant/family-plan está diseñado para aceptar mensajes de la familia, usar IA local cuando LLM_ASSISTANT_ENABLED=true, y degradar a fallback si el asistente falla.

El endpoint devuelve mode: "ai" o mode: "fallback", garantizando experiencia operativa incluso con servicio inaccesible.

8. Frontend y Experiencia de Usuario

Estado actual

El frontend es una SPA React responsive que consume exclusivamente VITE_API_URL, usa Zustand para estado y autenticación, implementa rutas protegidas por rol y tiene un playground de asistente en desarrollo.

Estrategia UX

- Priorizar decisiones rápidas para familias.
- Mostrar atributos de accesibilidad de forma inmediata.
- Explicar recomendaciones y modos de fallback.
- No exponer la complejidad de servicios externos al usuario.

Integración técnica

- apiClient.js gestiona la base URL del backend.
- recommendationsApi.js y assistantApi.js consumen el backend.
- recommendationsToCards mapea la respuesta a tarjetas de UI.
- El playground de chat usa POST /api/assistant/family-plan.

Puntos pendientes

- Alinear todas las pantallas con los endpoints reales.
- Eliminar gradualmente alias legacy de activity cuando el frontend esté listo.
- Completar la integración de favoritos y detalle de eventos.

9. IA, Asistente Conversacional e Integraciones Externas

Arquitectura de IA

La cadena de integración sigue el flujo: Frontend → Backend Express → ai-service / Data API. El frontend nunca llama directamente a servicios de IA. ai-service actúa como proxy local para Ollama y fuentes de datos externas.

Contrato del asistente

- LLM_ASSISTANT_ENABLED=false por defecto.
- LLM_ASSISTANT_API_URL y LLM_ASSISTANT_TIMEOUT_MS controlan la integración.

- LLM_ASSISTANT_CONTRACT permite seleccionar entre protocolos get-question y post-family-plan.

Integración de Open-Meteo y Euskadi

El prototipo de ai-service usa datos de Ollama para generación de lenguaje natural, APIs de eventos de Euskadi y Open-Meteo para contexto climático. Esta integración es experimental y se gestiona como capa de prototipo local.

Fallback y resiliencia

- Si ai-service no responde, el backend devuelve mode: "fallback".
- El modo fallback provee recomendaciones locales y mensajes amigables.
- La experiencia no depende exclusivamente de IA.



10. Seguridad, Privacidad y Compliance

Medidas aplicadas

- Cookies httpOnly para JWT.

- CORS restringido a CLIENT_URL.
- Variables de entorno documentadas en backend/.env.example.
- Validación de entradas en backend con express-validator.
- Manejo centralizado de errores.

Privacidad de menores

- FamilyMember almacena solo edad y nombre opcional.
- No se capturan datos sensibles innecesarios de menores.
- El proyecto reconoce el alcance de GDPR infantil y requiere un plan de compliance específico.

Riesgos de IA

- El prototipo en ai-service/ incluye código de prueba con eval() que no es seguro para producción.
- El asistente local debe permanecer en entornos controlados.
- ai-service nunca debe exponerse directamente al navegador.

Deuda de compliance

- Aún no hay tratamiento formal completo para GDPR infantil.
- La gestión de consentimiento y la política de privacidad están pendientes de implementación.
- Se deben documentar claramente los datos recogidos y el uso de IA.

11. Calidad, Pruebas y Metodología

Pruebas existentes

- Backend con Vitest y Supertest.
- Frontend con pruebas de servicios y mappers.
- Casos claros para recommendations, assistantApi y recommendationMapper.

Metodología

- KISS y MVP primero.
- Documentación de contrato API y auditorías en docs/.
- Branching policy y reglas de agente en AGENTS.md y CLAUDE.md.
- PRs guiados por revisión; no usar git add ..

Documentación relevante

► docs/api.md

- ▶ docs/database.md
- ▶ docs/integration-ai-ollama-local.md
- ▶ docs/features/backend-recommendations-data-normalization.md
- ▶ docs/contracts/frontend-backend-api-contract.md

12. Riesgos y Deuda Técnica

Riesgos técnicos

- Dependencia de la Data API externa y de servicios IA.
- Contrato legacy activity/event que aumenta la complejidad de mantenimiento.
- Riesgo de exposición de ai-service si se configura mal.
- Mapeo de datos desde APIs externas que puede fallar si un campo es nulo.

Deuda tecnológica

- Consolidar eventos reales como única entidad.
- Vaciar alias legacy cuando el frontend esté preparado.
- Mejorar la documentación de los flujos de error.
- Formalizar el tratamiento GDPR infantil.
- Asegurar la integración de servicios externos en producción.

13. Roadmap y Próximos Pasos

Prioridad de consolidación antes de producción:

Consolidar events como entidad única y retirar el alias activity.
Completar la integración frontend ↔ backend con el contrato actual.
Asegurar ai-service y eliminar eval() del prototipo.
Definir la gobernanza de datos de menores y el cumplimiento GDPR.
Añadir paneles de administración para moderación y gestión de eventos.
Versionar el esquema real y fijar el modelo de datos con ADR.

14. Referencias Internas

Documentación y fuentes de verdad del repositorio:

- ▶ README.md
- ▶ BACKEND_README.md
- ▶ docs/api.md
- ▶ docs/security.md
- ▶ docs/database.md
- ▶ docs/integration-ai-ollama-local.md
- ▶ docs/integration-frontend-backend.md
- ▶ docs/features/auth-roles-minimum.md
- ▶ docs/features/backend-recommendations-data-normalization.md
- ▶ docs/quality/README.md
- ▶ ai-service/README.md
- ▶ docs/contracts/frontend-backend-api-contract.md
- ▶ docs/data/schema-real/init.sql